

# InterviewPal — مستند پیاده‌سازی فرانت‌اند

اپلیکیشن تمرین مصاحبه فنی برای برنامه‌نویسان فارسی‌زبان

| مورد    | مقدار                                                                        |
|---------|------------------------------------------------------------------------------|
| فناوری  | TypeScript 6 و Angular 22 (standalone, zoneless, signals)                    |
| ریپو    | github.com/nasibehash/interviewPal-frontend                                  |
| اپ زنده | interview-pal-frontend-sable.vercel.app                                      |
| API     | interviewpal-backend.onrender.com (ASP.NET Core, ریپوی interviewPal-backend) |
| وضعیت   | فاز ۱ (MVP): بدون ورود به حساب؛ پیشرفت کاربر در مرورگر ذخیره می‌شود          |

## ۱. هدف و دامنه محصول

InterviewPal به برنامه‌نویس کمک می‌کند برای مصاحبه فنی تمرین کند. محتوا فارسی است و اصطلاح‌های فنی و کدها انگلیسی می‌مانند. فرانت‌اند دو بخش اصلی دارد:

- تمرین مصاحبه: ۳۰۰ سؤال در شش فناوری (TypeScript، React، Next.js، JavaScript، Angular، و NET) در سه سطح جونیور، مید و سنیور و سه حالت تمرین: یادگیری، مصاحبه (با تایمر) و فلش‌کارت.
- الگوریتم و الگوهای طراحی: ۲۴ درس (۱۴ الگوریتم و ۱۰ الگوی طراحی) که هر کدام مثال واقعی دارد و کدش به زبان یا فریم‌ورک انتخابی کاربر نمایش داده می‌شود، همراه با تمرین.
- پیشرفت من: تاریخچه تمرین‌ها، میانگین نمره، روزهای پشت‌سرهم و سؤال‌های ضعیف.

## ۲. پشته فناوری و تصمیم‌های اصلی

| حوزه        | انتخاب                                                               | دلیل                                                                   |
|-------------|----------------------------------------------------------------------|------------------------------------------------------------------------|
| فریم‌ورک    | Angular 22، کامپوننت‌های standalone                                  | آخرین نسخه؛ بدون NgModule و با بارگذاری تنبل ساده                      |
| تشخیص تغییر | Zoneless و OnPush (هر دو پیش‌فرض Angular 22)                         | بدون zone.js؛ همه state با signal است و فقط بخش تغییرکرده بررسی می‌شود |
| state       | signal و computed در سرویس‌های root                                  | بدون کتابخانه state جدا؛ جریان داده ساده و قابل‌تست                    |
| داده از API | httpResource (داخل ApiClient) برای خواندن، firstValueFrom برای نوشتن | خواندن‌ها reactive اند و با تغییر signal دوباره درخواست می‌شوند        |
| فرم‌ها      | Signal Forms (@angular/forms/signals)                                | مدل فرم یک signal ساده است؛ بدون ngModel و FormsModule                 |

| حوزه       | انتخاب                                                                   | دلیل                                                                |
|------------|--------------------------------------------------------------------------|---------------------------------------------------------------------|
| قالب       | کنترل فلو <code>@empty</code> / <code>@for</code> / <code>@if</code>     | سینتکس جدید Angular، بدون <code>*ngIf</code>                        |
| زبان و جهت | فارسی، <code>dir="rtl"</code> و خصوصیت‌های منطقی CSS                     | یک پایه برای RTL؛ کدها جداگانه LTR نمایش داده می‌شوند               |
| فونت       | Vazirmatn (بسته <code>@fontsource-variable/vazirmatn</code> )            | بدون درخواست به CDN بیرونی                                          |
| Markdown   | <code>marked</code> و <code>DOMPurify</code> و <code>highlight.js</code> | محتوای درس‌ها Markdown است؛ بعد از تبدیل پاک‌سازی (sanitize) می‌شود |
| تست        | Vitest از طریق <code>ng test</code>                                      | اجرای سریع و ساده؛ ۴۳ تست واحد                                      |
| استایل     | SCSS ساده با متغیرهای CSS و تم روشن/تیره                                 | بدون کتابخانه UI؛ حجم کم                                            |

### چرا ذخیره‌سازی در مرورگر؟

فاز ۱ ورود به حساب ندارد. تاریخچه و پیشرفت در `localStorage` و تمرین نیمه‌کاره در `sessionStorage` می‌ماند. همه خواندن و نوشتن‌ها از یک لایه امن (safe-storage) می‌گذرند تا در حالت مرور خصوصی یا ذخیره‌سازی مسدود، برنامه نشکند. وقتی حساب کاربری آمد، همین سرویس‌ها به API وصل می‌شوند و بقیه برنامه تغییر نمی‌کند.

## ۳. ساختار پروژه

```
src/app/
  app.ts / app.html / app.scss    shell: header, nav, router-outlet
  app.config.ts                  providers: http + interceptor, router features
  app.routes.ts                  lazy routes and guards
  core/                           logic and data, no UI
    models.ts                    TypeScript types of the API contract
    api-client.ts                 the only place that talks HTTP (resources + calls)
    api-error.ts                  interceptor: HTTP failure -> ApiError (Persian message)
    practice-session.ts           state of the practice in progress
    progress-store.ts             history and stats (localStorage)
    lesson-progress.ts            progress of lesson exercises
    preferred-technology.ts        the language the learner picked
    self-assessment.ts            "I knew it" -> a gradable answer
    safe-storage.ts               storage reads/writes that never throw
  shared/                         shared components (ts + html + scss each)
    markdown.ts, markdown.pipe.ts, question-view, answer-panel,
    report-dialog, code-block
  features/                       pages, each one lazy loaded
    setup/ practice/ result/ history/ lessons/
  docs/
    frontend.md                  this document (source)
    InterviewPal-Frontend.pdf    the same document as PDF
```

در این ساختار، هر پوشه یک مسئولیت دارد: `core` منطق و داده، `shared` اجزای تکراری و `features` صفحه‌ها.

## قرارداد نام‌گذاری فایل‌ها

از Angular 20 به بعد راهنمای رسمی سبک (style guide) و دستور `ng generate` پسوند `.service` / `.component` را حذف کرده‌اند. به جای `lesson.component.ts` نام فایل `lesson.ts` و نام کلاس `Lesson` است (نه `LessonComponent`). این پروژه همین قرارداد را دنبال می‌کند:

- هر کامپوننت سه فایل جدا دارد: `name.ts`، `name.html` و `name.scss` (با `templateUrl` و `styleUrl`). قالب و استایل `inline` فقط در تست‌ها استفاده می‌شود.
- نام صفحه‌ها پسوند `-page` دارد (`lesson-page.ts`) تا از کامپوننت‌های کوچک‌تر در پوشه `shared` جدا شوند؛ این یک انتخاب پروژه است، نه الزام Angular.
- سرویس‌ها بدون پسوند و با نام معنادار: `ApiClient`، `ProgressStore`.
- قاعده اصلی: `core` به هیچ کامپوننتی وابسته نیست و `features` فقط از `core` و `shared` استفاده می‌کنند. هر صفحه با `loadComponent` بارگذاری می‌شود و `bundle` اولیه کوچک می‌ماند (حدود ۹۲ کیلوبایت فشرده).

## ۴. مسیریابی

| مسیر         | صفحه         | توضیح                                                                                                   |
|--------------|--------------|---------------------------------------------------------------------------------------------------------|
| /            | شروع تمرین   | انتخاب تکنولوژی، سطح، تعداد (۵ تا ۱۰۰) و حالت                                                           |
| /practice    | تمرین        | guard: فقط وقتی تمرین فعال وجود دارد                                                                    |
| /result      | نتیجه        | guard: فقط بعد از تمام شدن و ارزیابی                                                                    |
| /lessons     | فهرست درس‌ها | تب الگوریتم‌ها و الگوهای طراحی، فیلتر سطح                                                               |
| /lessons/:id | درس          | پارامتر <code>id</code> با <code>withComponentInputBinding</code> مستقیم به <code>input()</code> می‌رسد |
| /history     | پیشرفت من    | آمار و تاریخچه                                                                                          |
| **           | —            | بازگشت به صفحه اصلی                                                                                     |

guardها تابعی (`CanActivateFn`) هستند و فقط وضعیت `PracticeSessionStore` را می‌خوانند: اگر کاربر مستقیم `/practice` را باز کند و تمرینی نداشته باشد، به `/` هدایت می‌شود.

## ۵. لایه core

### ۵.۱ مدل‌ها و ApiClient

`models.ts` آیینة قرارداد JSON بک‌اند است (`camelCase` و `enum` به صورت رشته). سؤال‌ها بدون جواب می‌آیند و جواب فقط بعد از پاسخ‌دادن از API گرفته می‌شود، پس جواب در `payload` اولیه نیست. `ApiClient` تنها جایی است که `HttpClient` را می‌شناسد و آدرس پایه از توکن `API_BASE_URL` (پیش‌فرض `/api`) می‌آید. دو دسته متد دارد:

- خواندن به صورت `resource`: `lessonResource(id, lessonsResource(), technologiesResource())` یک `technology` `httpResource` برمی گردانند. صفحه ها آن را در `field initializer` می سازند و فقط `value()`، `isLoading()`، `error()` و `reload()` را می بینند؛ هیچ صفحه ای URL نمی سازد. `lessonResource` دو تابع (برای `id` و زبان) می گیرد و با تغییر هر کدام خودش دوباره درخواست می زند.
- نوشتن و بقیه به صورت `Observable`: ارزیابی پاسخ، ساخت تمرین و گزارش، که در state با `firstValueFrom` مصرف می شوند.
- `apiErrorInterceptor` (فایل `api-error.ts`) هر خطای HTTP را به `ApiError` با پیام فارسی تبدیل می کند: « (شبکه)، ۴۰۰ (متن ProblemDetails سرور)، ۴۰۴، ۴۲۹ و ۵xx. بنابراین store ها و صفحه ها هیچ وقت `HttpErrorResponse` خام نمی بینند.

## ۵.۲ PracticeSessionStore

- قلب بخش تمرین است و کل state را با signal نگه می دارد: `session`، `answers`، `revealed`، `correctness`، `notes`، `index`، `startedAt`، `phase`، `evaluation`، `busy` و `error`. فاز تمرین یکی از مقدارهای زیر است:

```
idle -> practicing -> (self-assessment) -> submitting -> finished
      ^                               |
      +----- evaluation failed -----+
```

- حالت یادگیری: با انتخاب هر گزینه، پاسخ فوراً به API (`check`) فرستاده می شود و درست/غلط و توضیح نشان داده می شود؛ پاسخ قفل می شود.
- حالت مصاحبه: پاسخ ها فقط در حافظه جمع می شوند و در پایان با یک درخواست (`evaluate`) ارزیابی می شوند. سؤال های تشریحی که کاربر خودارزیابی نکرده، قبل از ارسال وارد فاز `self-assessment` می شوند.
- حالت فلش کارت: جواب نمایش داده می شود و کاربر خودش «بلد بودم / بلد نبودم» می زند.
- سؤال بی جواب در انتها غلط حساب می شود (`completeAnswers`).
- تمرین نیمه کاره در `sessionStorage` ذخیره می شود و با `refresh` از همان سؤال ادامه پیدا می کند (با `effect`).
- اگر ارزیابی با خطا روبه رو شود، فاز به `practicing` برمی گردد تا کاربر دوباره تلاش کند.

تابع `answerFromSelfAssessment` نتیجه خودارزیابی را به پاسخی تبدیل می کند که API بفهمد: برای سؤال تشریحی `knewIt` و برای سؤال گزینه ای، گزینه درست (اگر کاربر بلد بوده) یا یک گزینه غلط.

## ۵.۳ ProgressStore و دیگر سرویس ها

| سرویس                | کلید storage                                                                       | کار                                                                             |
|----------------------|------------------------------------------------------------------------------------|---------------------------------------------------------------------------------|
| ProgressStore        | <code>interviewpal.history.v1</code> و <code>interviewpal.question-stats.v1</code> | حداکثر ۱۰۰ تمرین اخیر؛ آمار هر سؤال؛ میانگین، streak و سؤال های ضعیف (computed) |
| PracticeSessionStore | <code>interviewpal.active-session.v1</code> (sessionStorage)                       | تمرین در حال انجام                                                              |

| سرویس               | کلید storage                                 | کار                                                                                           |
|---------------------|----------------------------------------------|-----------------------------------------------------------------------------------------------|
| LessonProgress      | <code>interviewpal.lesson-progress.v2</code> | پاسخ آخر هر تمرین درس؛ درس وقتی کامل است که همه تمرین‌ها پاسخ داده و آخرین پاسخ‌ها درست باشند |
| PreferredTechnology | <code>interviewpal.technology.v1</code>      | زبان انتخابی؛ مقدار نامعتبر به <code>javascript</code> برمی‌گردد                              |

نسخه ( `.v1` ، `.v2` ) در نام کلیدها گذاشته شده تا اگر ساختار داده عوض شد، داده‌های قدیمی بی‌صدا نادیده گرفته شوند و برنامه نشکند.

## ۶. صفحه‌ها

### ۶.۱ شروع تمرین (setup)

فهرست تکنولوژی‌ها با `httpResource` از `/technologies` می‌آید و حالت‌های `loading` و خطا (با دکمه «تلاش دوباره») دارد. انتخاب تکنولوژی و سطح چندگانه است و خالی بودن یعنی «همه». تعداد سؤال با اسلایدر ۵ تا ۱۰۰ و زمان تقریبی (حدود ۸۰ ثانیه برای هر سؤال) نشان داده می‌شود.

### ۶.۲ تمرین (practice)

- نوار پیشرفت و شمارنده «سؤال n از N»؛ دکمه‌های قبلی/بعدی و «پایان تمرین» در سؤال آخر.
- در حالت مصاحبه تایمر معکوس از `timeLimitSeconds` نشان داده می‌شود؛ زیر یک دقیقه قرمز می‌شود و با صفر شدن، تمرین خودکار ارسال می‌شود.
- گزینه‌ها بعد از پاسخ به رنگ درست (سبز) و غلط (قرمز) درمی‌آیند و پاسخ کامل (جواب کوتاه، توضیح، اشتباه رایج و سؤال بعدی مصاحبه‌کننده) نمایش داده می‌شود.
- گزارش مشکل یک سؤال ( `report-dialog` ) بعد از دیدن جواب در دسترس است.

### ۶.۳ نتیجه (result)

درصد و تعداد پاسخ درست، نمودار میله‌ای به تفکیک تکنولوژی و سطح، موضوع‌های ضعیف (tag) و مرور همه سؤال‌ها با توضیح. دکمه «تمرین دوباره با سؤال‌های ضعیف» با `startFromQuestions` یک تمرین یادگیری از همان سؤال‌ها می‌سازد.

### ۶.۴ پیشرفت من (history)

تعداد تمرین، میانگین نمره، روزهای پشت‌سرهم، دقت هر تکنولوژی، فهرست سؤال‌های ضعیف و ۱۵ تمرین آخر. دکمه پاک‌کردن سابقه بعد از تأیید کاربر همه داده‌ها را حذف می‌کند.

### ۶.۵ درس‌ها (lessons)

- فهرست: انتخاب زبان در بالای صفحه، دو تب (الگوریتم‌ها / الگوهای طراحی)، فیلتر سطح، گروه‌بندی بر اساس دسته و نشان «✓ کامل» روی درس‌های تمام‌شده.

- جست‌وجو: یک Signal Form با `debounce` ۲۵۰ میلی‌ثانیه؛ در عنوان، خلاصه و تگ‌ها می‌گردد و حروف عربی و فارسی («ك»/«ک» و «ي»/«ی») را یکی می‌گیرد.
- صفحهٔ درس: مثال واقعی، توضیح، پیچیدگی (برای الگوریتم‌ها)، کد به زبان انتخابی با دکمهٔ کپی، شرح کد، «کی استفاده کنم / کی نه»، اشتباه رایج و ۴ تمرین.
- عوض کردن زبان در وسط درس، درس را با `technology` جدید دوباره از API می‌گیرد؛ چون `httpResource` به `signal` زبان وابسته است.
- بخش تمرین‌ها داخل `@defer (on viewport)` است: کد آن (کامپوننت `LessonExerciseView`) در یک `chunk` جدا بارگذاری می‌شود و فقط وقتی کاربر به پایین صفحه برسد.
- هر تمرین جدا با API بررسی می‌شود (جواب درست در `payload` درس نیست) و نتیجه در `LessonProgress` ثبت می‌شود.

## ۷. کامپوننت‌های مشترک

| کامپوننت                   | کار                                                                                                                                                                                                                                 |
|----------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| markdown.ts و MarkdownPipe | تبدیل Markdown به HTML، رنگ‌آمیزی کد با <code>highlight.js</code> (فقط زبان‌های لازم ثبت شده‌اند: JavaScript، TypeScript، HTML، C#، JSON، bash، CSS و فقط بعد از آن <code>DOMPurify</code> و <code>bypassSecurityTrustHtml</code> ) |
| QuestionView               | متن سؤال، سطح (رنگی)، تگ‌ها و قطعهٔ کد (همیشه <code>dir="ltr"</code> )                                                                                                                                                              |
| AnswerPanel                | جواب کوتاه، توضیح کامل، اشتباه رایج و سؤال بعدی                                                                                                                                                                                     |
| ReportDialog               | گزارش جواب غلط، قدیمی، مبهم یا غلط نگارشی؛ با Signal Form، <code>required</code> ، <code>maxLength(500)</code> و <code>&lt;form [formRoot]&gt;</code>                                                                               |
| CodeBlock                  | نمایش کد با زبان و دکمهٔ «کپی‌کد» (clipboard)؛ خروجی <code>highlight.js</code> خودش کد را <code>escape</code> می‌کند                                                                                                                |

### امنیت

همهٔ متن‌های Markdown (سؤال، توضیح، درس) قبل از رسیدن به DOM با `DOMPurify` پاک‌سازی می‌شوند. اسکریپت و `onerror` حذف می‌شود و این رفتار تست واحد دارد. هیچ `innerHTML` دیگری بدون این مسیر استفاده نشده است.

## ۸. ویژگی‌های Angular 22 که در پروژه استفاده شده

| ویژگی                       | کجا                                                         | چرا                                                                                          |
|-----------------------------|-------------------------------------------------------------|----------------------------------------------------------------------------------------------|
| Zoneless (پیش‌فرض)          | <code>bootstrapApplication</code> بدون <code>zone.js</code> | رندر فقط با تغییر <code>signal</code> ؛ بدون <code>patch</code> کردن API‌های مرورگر          |
| <code>OnPush</code> پیش‌فرض | همهٔ کامپوننت‌ها (بدون تنظیم صریح)                          | در Angular 22 پیش‌فرض است؛ همهٔ <code>state</code> با <code>signal</code> است، پس سازگار است |

| ویژگی                         | کجا                                                                         | چرا                                                                                                                                                     |
|-------------------------------|-----------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------|
| httpResource (پایدار از 22.0) | ApiClient.technologiesResource() ، lessonResource() ، lessonsResource()     | خواندن reactive: با تغییر id یا زبان خودش دوباره درخواست می‌زند و reload / error / isLoading می‌دهد؛ defaultValue: [] الگوی [] ?? را از قالب‌ها حذف کرد |
| Signal Forms (پایدار از 22.0) | ReportDialog و جست‌وجوی LessonsPage                                         | <form ، [formField] ، form() ، submission با [formRoot]> ؛ debounce ، maxLength ، required ؛ مدل فرم یک signal ساده است                                 |
| linkedSignal                  | LessonExerciseView                                                          | state قابل‌نوشتن که با عوض شدن exercise ورودی خودش بازنشانی می‌شود                                                                                      |
| @defer (on viewport)          | تمرین‌های LessonPage                                                        | کد تمرین‌ها در یک chunk جدا و فقط با رسیدن به پایین صفحه بارگذاری می‌شود                                                                                |
| @let                          | LessonPage                                                                  | متغیر محلی در قالب، به جای تکرار technology.slug()                                                                                                      |
| کنترل‌فلو جدید                | همه قالب‌ها                                                                 | @empty ، track با @for ، @if                                                                                                                            |
| / input() input.required()    | کامپوننت‌ها و LessonPage                                                    | ورودی‌های signal؛ با withComponentInputBinding پارامتر id: مسیر مستقیم به input می‌رسد                                                                  |
| computed و effect             | storeها و صفحه‌ها                                                           | مشتق‌های خالص با computed ؛ effect فقط برای هماهنگ‌سازی با sessionStorage و تایمر                                                                       |
| Functional interceptor        | apiErrorInterceptor با withInterceptors                                     | تبدیل خطاها به ApiError در یک نقطه                                                                                                                      |
| Router                        | withViewTransitions ، withInMemoryScrolling loadComponent ، guardهای تابعی، | انتقال انیمیشنی بین صفحه‌ها، برگرداندن اسکرول و bundle اولیه کوچک                                                                                       |
| inject()                      | همه جا                                                                      | بدون constructor injection؛ کار با field initializer و تابع‌های کمکی ساده می‌شود                                                                        |
| Vitest با ng test             | همه تست‌ها                                                                  | @angular/build:unit-test builder بدون Karma و Jasmine                                                                                                   |

بعضی از این‌ها، مثل httpResource و Signal Forms، در نسخه‌های قبلی آزمایشی بودند. در Angular 22 هر دو در تایپ‌های خود فریم‌ورک با @publicApi 22.0 (یعنی API عمومی و پایدار) علامت خورده‌اند.

## ۹. استایل، RTL و دسترس پذیری

- متغیرهای CSS در `root` و تم تیره با `prefers-color-scheme`؛ رنگ اصلی بنفش (`#4F46E5`) و رنگ های جدا برای موفقیت، خطا و هشدار.
- خصوصیت های منطقی (`inset-block-start`، `inline-size`، `margin-inline`) به جای `left/right` تا جهت RTL خودکار درست شود.
- کد و عبارت های انگلیسی `dir="ltr"` و `unicode-bidi: plaintext` دارند تا در متن فارسی به هم نریزند.
- دکمه های انتخابی `aria-pressed`، تب ها `role="tab"` و `aria-selected`، تایمر `role="timer"`، خطاها `role="alert"` و نوار پیشرفت `role="progressbar"` دارند. `focus` قابل مشاهده است.
- `prefers-reduced-motion` انیمیشن ها را خاموش می کند و طرح روی عرض موبایل بدون اسکرول افقی کار می کند.

## ۱۰. قرارداد API

| متد  | مسیر                                                        | استفاده                                        |
|------|-------------------------------------------------------------|------------------------------------------------|
| GET  | <code>/api/technologies</code>                              | فهرست تکنولوژی ها و تعداد سؤال هر سطح          |
| POST | <code>/api/practice/sessions</code>                         | ساخت تمرین (تکنولوژی ها، سطح ها، تعداد، حالت)  |
| POST | <code>/api/practice/questions/{id}/check</code>             | ارزیابی یک پاسخ (یادگیری)                      |
| POST | <code>/api/practice/evaluate</code>                         | ارزیابی کل تمرین (مصاحبه)                      |
| GET  | <code>/api/questions/{id}</code>                            | یک سؤال با جواب (برای تمرین دوباره و فلش کارت) |
| POST | <code>/api/questions/{id}/reports</code>                    | گزارش مشکل سؤال                                |
| GET  | <code>/api/lessons?kind=</code>                             | فهرست درس ها                                   |
| GET  | <code>/api/lessons/{id}?technology=</code>                  | درس با کد زبان انتخابی                         |
| POST | <code>/api/lessons/{id}/exercises/{exerciseId}/check</code> | بررسی یک تمرین درس                             |

خطاها به صورت `ProblemDetails` می آیند. `PracticeSessionStore` پیام فارسی عمومی نشان می دهد و صفحه ها برای خطای بارگذاری دکمه «تلاش دوباره» دارند.

## ۱۱. تست

۴۳ تست واحد با `Vitest` (`npm test`):

- `ApiClient`: مسیر و بدنه هر درخواست با `HttpTestingController`.
- `PracticeSessionStore`: شروع تمرین، ارزیابی فوری در یادگیری، نبودن درخواست در مصاحبه، فاز خودارزیابی، ارسال نهایی، بازگشت بعد از خطا و تمرین خالی.

- LessonProgress، ProgressStore، PreferredTechnology: ثبت، ماندگاری، پاک کردن، streak و مقدار نامعتبر storage.
- `apiErrorInterceptor`: تبدیل خطاهای HTTP به `ApiError` با پیام فارسی.
- `ReportDialog` (Signal Form): ارسال دلیل و پیام و جلوگیری از ارسال پیام بیش از ۵۰۰ کاراکتر.
- `LessonExerciseView`: بازنشانی پاسخ قبلی با `linkedSignal` وقتی تمرین عوض می‌شود.
- markdown: رنگ‌آمیزی، حذف اسکریپت و `onerror`، escape زبان‌های ناشناخته.
- LessonPage، SetupPage و LessonPage: لیست، فیلتر، جست‌وجوی debounce شده با حروف عربی/فارسی، انتخاب زبان، گرفتن درس با زبان جدید و بررسی تمرین. بلوک `@defer` در تست با `DeferBlockBehavior.Manual` دستی رندر می‌شود، چون jsdom از `IntersectionObserver` پشتیبانی نمی‌کند.
- علاوه بر آن، کل جریان یادگیری یک بار در مرورگر واقعی (Chromium) روی بک‌اند واقعی اجرا و بدون خطای کنسول تأیید شده است.

## ۱۲. اجرا و استقرار

### ۱۲.۱ توسعه

```
npm install
npm start      # http://localhost:4200 (/api به http://localhost:5161 می‌شود)
npm test
npm run build  # خروجی: dist/interviewpal/browser
```

Angular CLI 22 به Node نسخه ۲۲/۲۲ یا بالاتر نیاز دارد (Node 24 پیشنهاد می‌شود). بک‌اند باید روی `http://localhost:5161` در حال اجرا باشد (`proxy.conf.json`).

### ۱۲.۲ Docker

`Dockerfile` دو مرحله دارد: build با Node 24 و سرو با `nginx.conf.template`. مسیرهای Angular را به `index.html` برمی‌گرداند، فایل‌های hash را یک سال کش می‌کند، `index.html` را کش نمی‌کند و `/api` را به `API_URL` پراکسی می‌کند. `compose.yaml` بک‌اند و فرانت را کنار هم روی `http://localhost:8080` بالا می‌آورد.

### ۱۲.۳ Render و Vercel

- فرانت‌اند روی Vercel: preset Angular، دستور `npm run build`، خروجی `dist/interviewpal/browser` و Node 24.x.
- `vercel.json` مسیر `/api/*` را به `https://interviewpal-backend.onrender.com` می‌فرستد و بقیه مسیرها را به `index.html`؛ بنابراین مرورگر فقط یک origin می‌بیند و CORS لازم نیست.
- بک‌اند روی Render با Docker اجرا می‌شود. در پلن رایگان پس از ۱۵ دقیقه بی‌کاری می‌خوابد و اولین درخواست بعدی حدود نیم دقیقه طول می‌کشد.

## ۱۲.۴ مستندات

این سند در [docs/frontend.md](#) نوشته می‌شود و [docs/InterviewPal-Frontend.pdf](#) از روی آن ساخته می‌شود. با هر تغییر مهم، README، همین سند و PDF با هم به‌روز می‌شوند.

## ۱۳. محدودیت‌ها و گام‌های بعدی

| موضوع                 | وضعیت فعلی                       | گام بعدی                                             |
|-----------------------|----------------------------------|------------------------------------------------------|
| حساب کاربری           | ندارد؛ پیشرفت فقط در همان مرورگر | ورود، ذخیره تاریخچه روی سرور و همگام‌سازی چند دستگاه |
| حالت مصاحبه‌ زمان‌دار | تایمر ساده سمت کلاینت            | محاسبه زمان در سرور برای جلوگیری از دستکاری          |
| مرور فاصله‌دار        | فقط فهرست سؤال‌های ضعیف          | زمان‌بندی مرور بر اساس منحنی فراموشی                 |
| پنل ادمین             | محتوا با فایل JSON ویرایش می‌شود | پنل مدیریت سؤال و درس و بررسی گزارش‌ها               |
| مصاحبه‌کننده هوشمند   | ندارد                            | پاسخ متنی/صوتی و بازخورد                             |
| خواب بک‌اند رایگان    | تأخیر اولین درخواست              | پلن بدون خواب یا پینگ دوره‌ای                        |